

Allow defaulting comparisons by value

Document #: P1946R0
Date: 2019-11-08
Project: Programming Language C++
EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Casey Carter
<casey@carter.com>

§ 1 Introduction and Motivation

This is currently ill-formed:

```
struct C {  
    int i;  
    friend bool operator==(C, C) = default;  
};
```

Because the rule for defaulted comparisons in [class.compare.default] requires that a defaulted comparison for a class C be either:

- a non-static const member of C having one parameter of type `const C` `&`, or
- a friend of C having two parameters of type `const C` `&`.

There is no option for having two parameters of type C. This case appears not to have been previously discussed by Evolution, did not appear in any previous comparison papers, though was sort of mentioned in the Kona 2017 discussion of [P0515R0] that taking by-value was intended.

From Casey:

I've been using C++20 `==` and `<=>` quite a bit implementing test cases for Ranges things, and I can say that it's jarring to implement a class that's passed by value everywhere *except* for defaulted `==` and/or `<=>`. I'd like to see [class.compare.default]/1.1 changed as well to allow "a non-static const member of C having one parameter of type `const C` `&` or C, or". I can't see any particular reason to forbid pass-by-value for member or non-member defaulted comparison operators.

This seems like Core issue material to us, but just in case we're writing this paper to fix this minor defect, as we would rather fix it in the language than have to deal with fixing it in the library (as in [LWG3295]) and the inevitable user questions that will come up when this doesn't work.

There is a certain class of type that should be taken by value instead of by reference to `const` - and those situations surely include that type's own comparison operators.

§ 2 Evolution Direction

A draft of this paper was presented to Evolution in Belfast, which decided that they wanted to allow the friend by-value case but none of the mixed-`const` cases. In other words, allow A but disallow B and C below:

```
struct A {  
    friend bool operator==(A, A) = default;  
};  
  
struct B {  
    bool operator==(B) const = default;  
};  
  
struct C {  
    friend bool operator==(C, C const&) = default;  
};
```

§ 3 Wording

Change 11.1.1 [class.compare.default], paragraph 1:

- 1 A defaulted comparison operator function ([`expr.spaceship`], [`expr.rel`], [`expr.eq`]) for some class `C` shall be a non-template function declared in the *member-specification* of `C` that is
 - (1.1) — a non-static `const` member of `C` having one parameter of type `const C` `&`, or
 - (1.2) — a friend of `C` having **either** two parameters of type `const C` `&` **or two** parameters of type `C`.

§ 4 References

[LWG3295] Barry Revzin. Comparison category `operator==` are mis-specified.

<https://wg21.link/lwg3295>

[P0515R0] Herb Sutter. 2017. Consistent comparison.

<https://wg21.link/p0515r0>